Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015119**	**Course Name:**	**Agile SoftwareDevelopment& DevOps Lab**

Name of Student:	**Chirag Raut**
Roll No. :	85
Experiment No.:	**04**
Title of the Experiment:	**To containerize applications using Docker.**
Date of Performance:	
Date of Submission:	

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Ms. Kshitija Gharat**

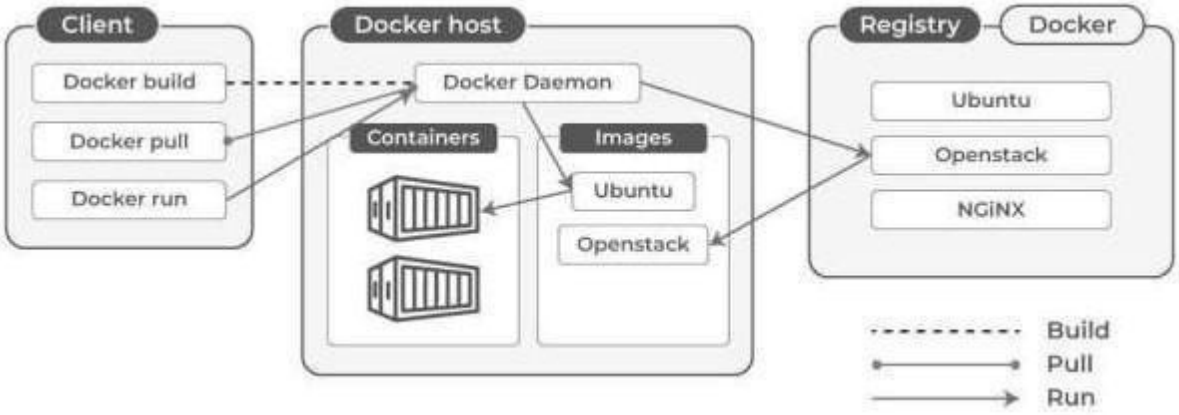
Signature :

Date :

Aim: To containerize applications using Docker

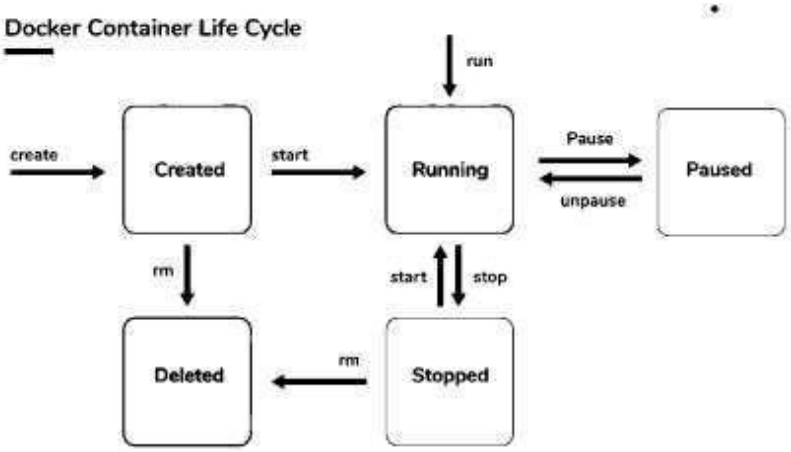
Objective: To understand Docker architecture and the lifecycle of containers and to learn essential Docker commands for managing images and interacting with containers. This enables users to package, distribute, and run applications consistently across different environments.

Theory: Docker is a containerization technology that allows developers to package their applications and dependencies into lightweight, portable containers. These containers can then be run on any system that supports Docker, making it easier to deploy and manage applications in various environments.
Docker Architecture:



The Docker architecture consists of the following components:
Docker Client: This is the primary interface that developers use to interact with Docker. It sends commands to the Docker daemon and displays the output.
Docker Daemon: This is the background process that runs on the host machine and manages Docker objects such as images, containers, networks, and volumes.
Docker Registry: This is a storage and distribution system for Docker images. It allows developers to share and distribute their images with others.
Docker Image: This is a lightweight, standalone, executable package that includes everything needed to run an application, including the application code, runtime, libraries, dependencies, and system tools.
Docker Container: This is a runtime instance of a Docker image. It is isolated from the host system and other containers, providing a secure and predictable environment for the application to run in.

Container LifeCycle:



The lifecycle of a Docker container consists of the following stages:

Create: To create a container, you start by creating an image that includes all the necessary components to run the application. This image is then used to create the container.

Start: Once the container is created, you can start it using the docker start command. This launches the container and runs the application inside it.

Run: Once the container is started, you can interact with the application inside the container. You can run commands, access files, and make changes to the application.

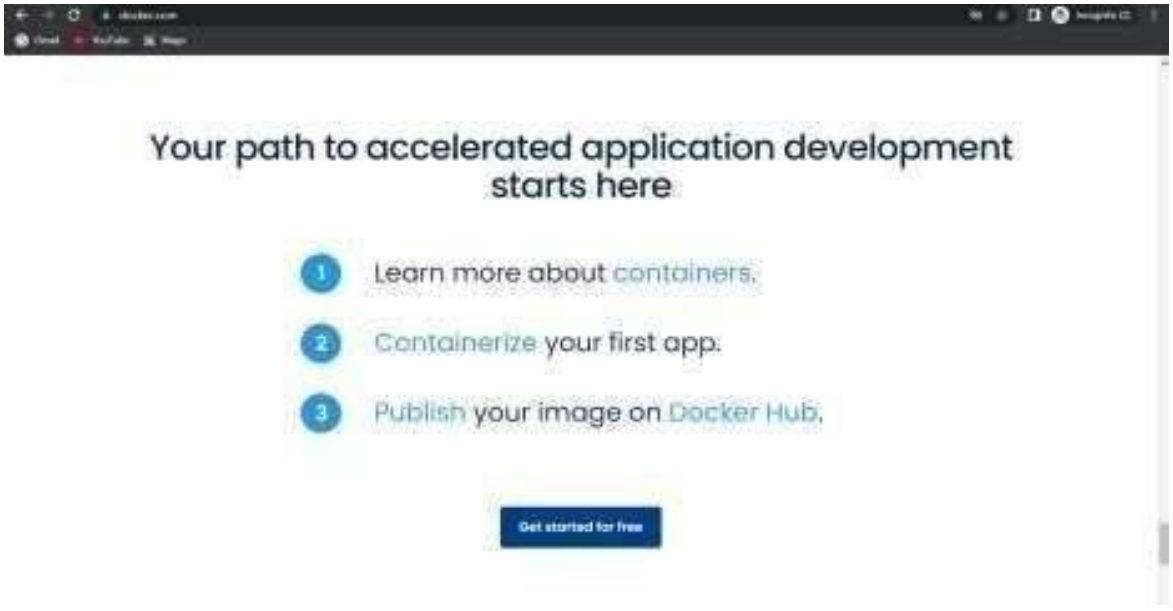
Stop: When you're done using the container, you can stop it using the docker stop command. This stops the application inside the container and shuts down the container.

Remove: Finally, when you no longer need the container, you can remove it using the docker rm command. This removes the container from the system and frees up any resources it was using.

Steps for Installation:

Step 01: Open docker.com

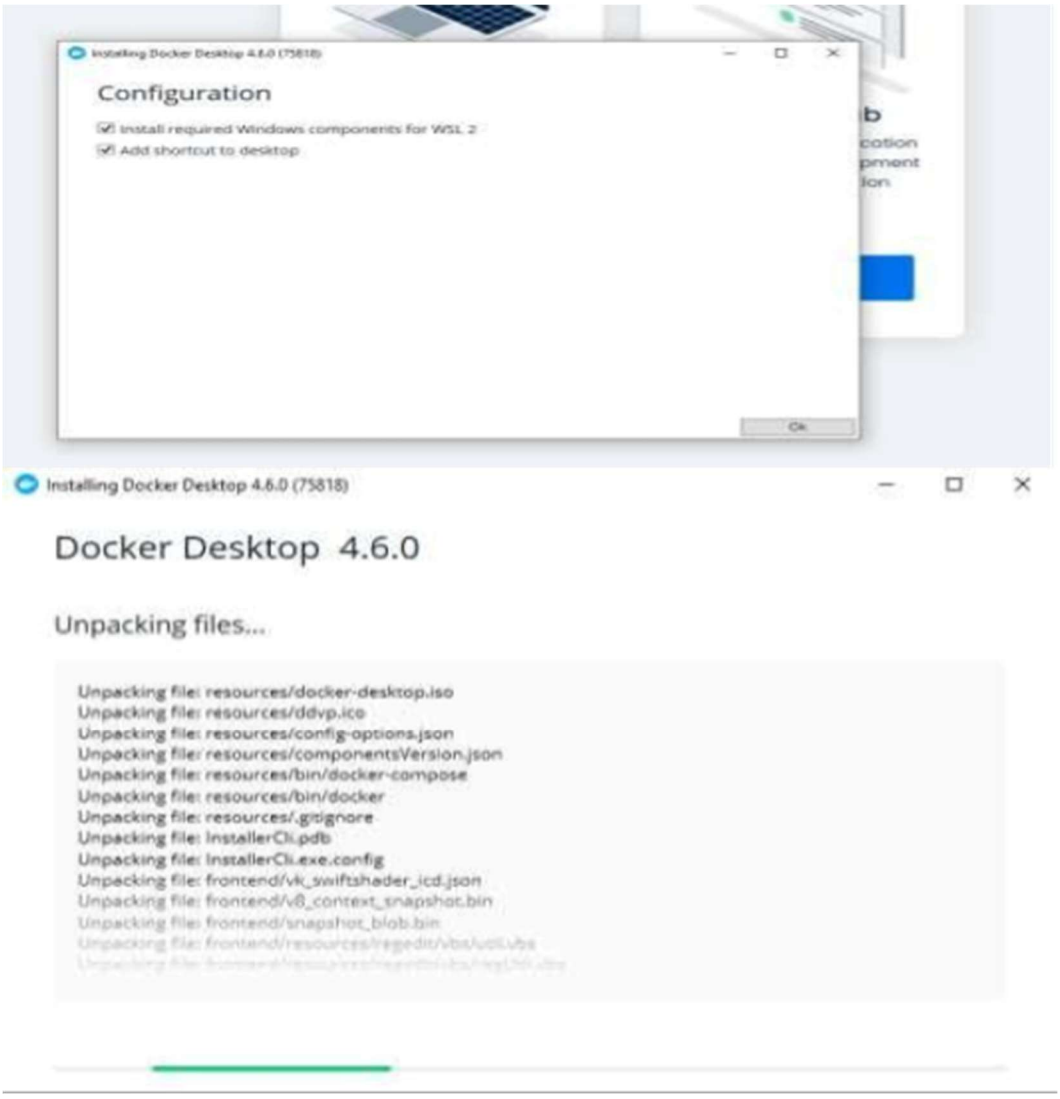
Scroll down, Click on 'Get started for free' tab.



Step 02: Click on Docker Desktop, Download it.



Step 03: After downloading, Open 'Docker Desktop Installer' & start installation.

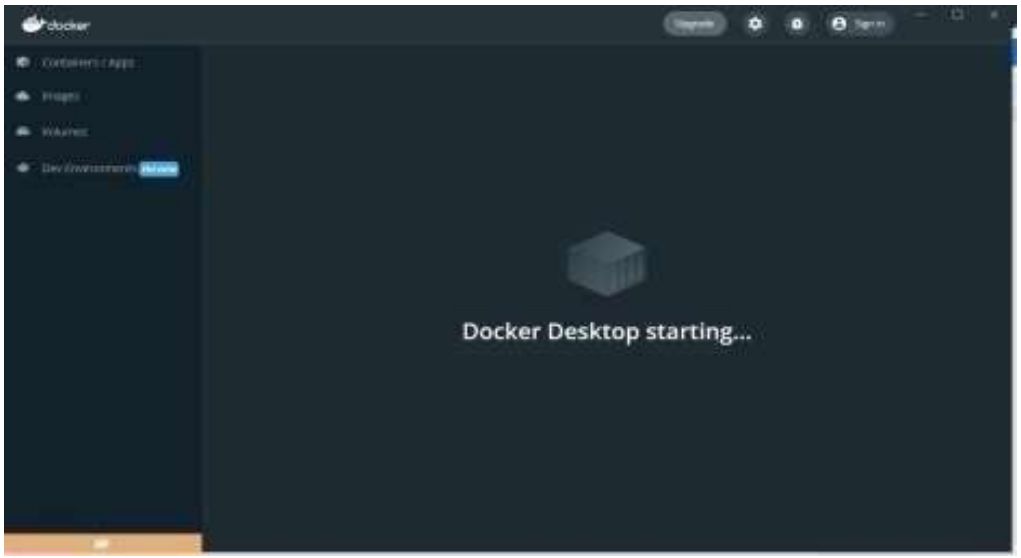


Step 04: After Installation, Restart your device.



Step 05: Accept the terms and conditions, Click on Accept.



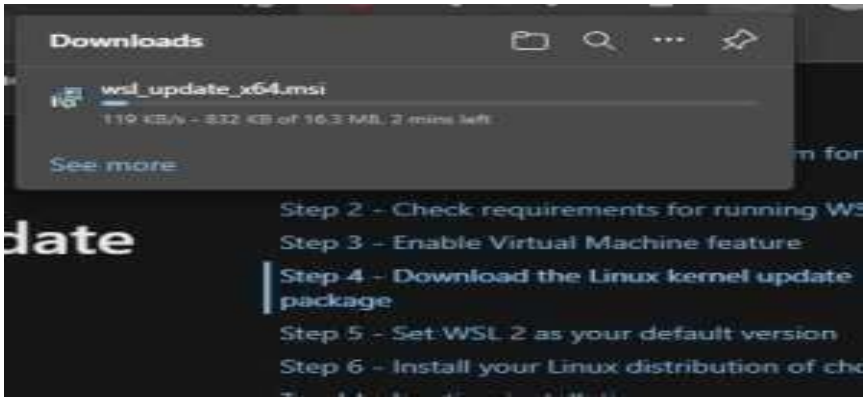


The following window should pop up.
Click on the link - https://aka.ms/wsl2kernel. (Do not close this window).

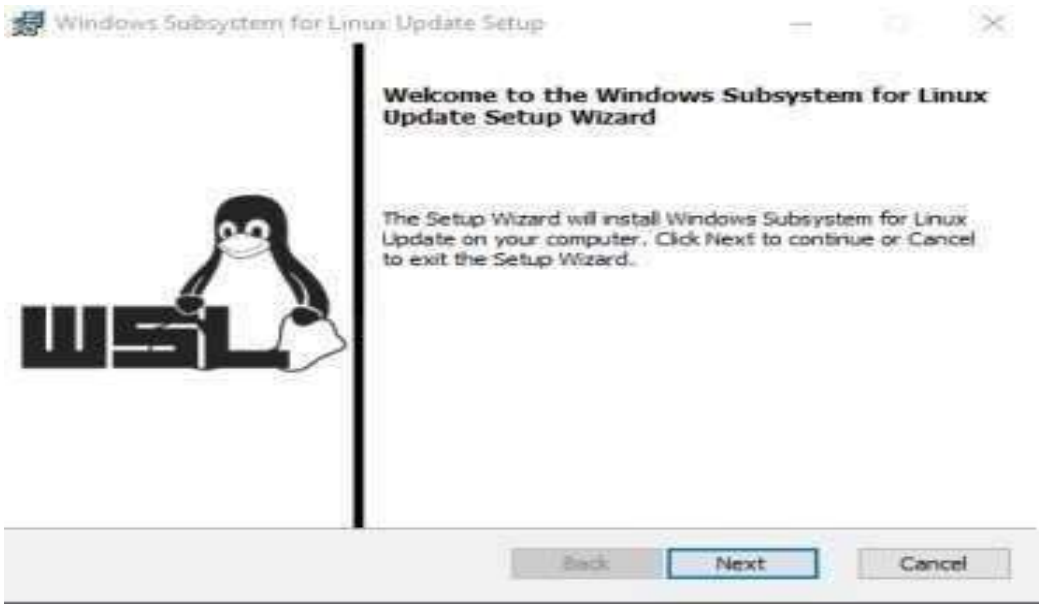


Download the WSL2 Linux kernel update package for x64 machines.

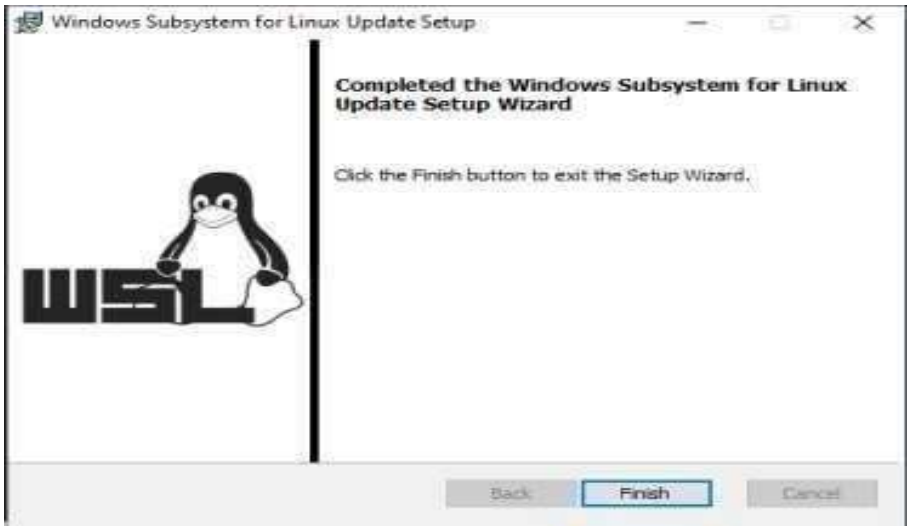




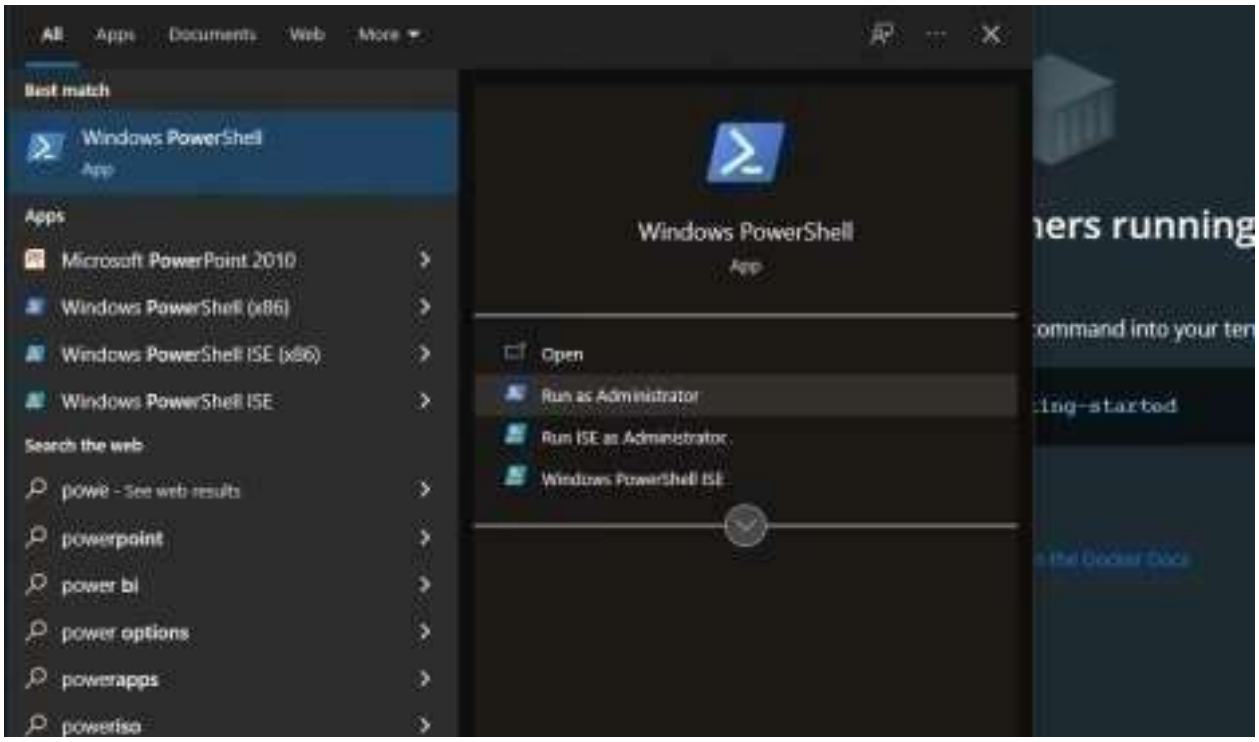
After Download is complete, Run the .msipackage. Click on next.



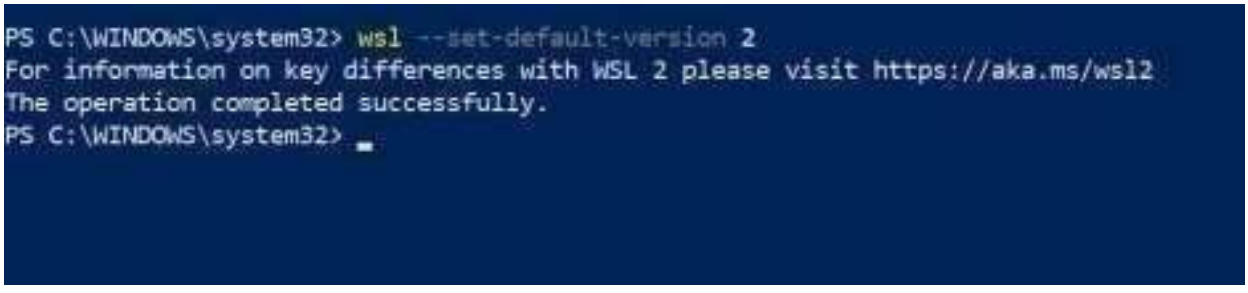
After, the setup is complete, Click on finish.



Open Powershell as an Administrator.



Run the following Command: wsl
--set-default-version 2



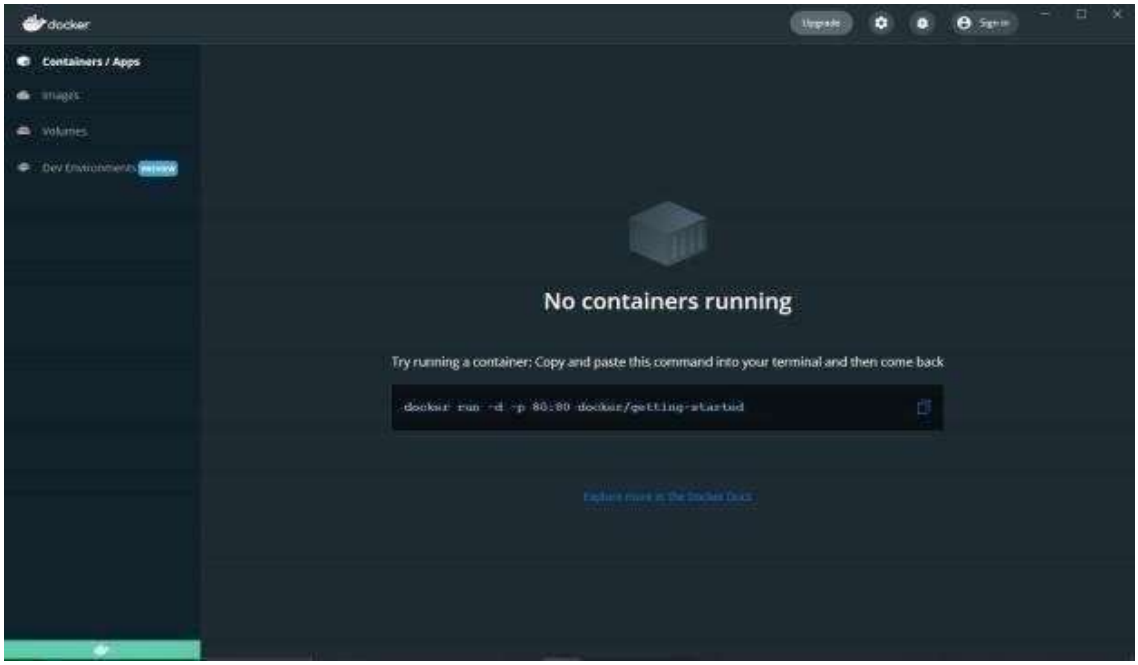
Now, Click on Restart



Docker should now restart. Click on Start.



The following window should pop up. This means, Installation is now complete.

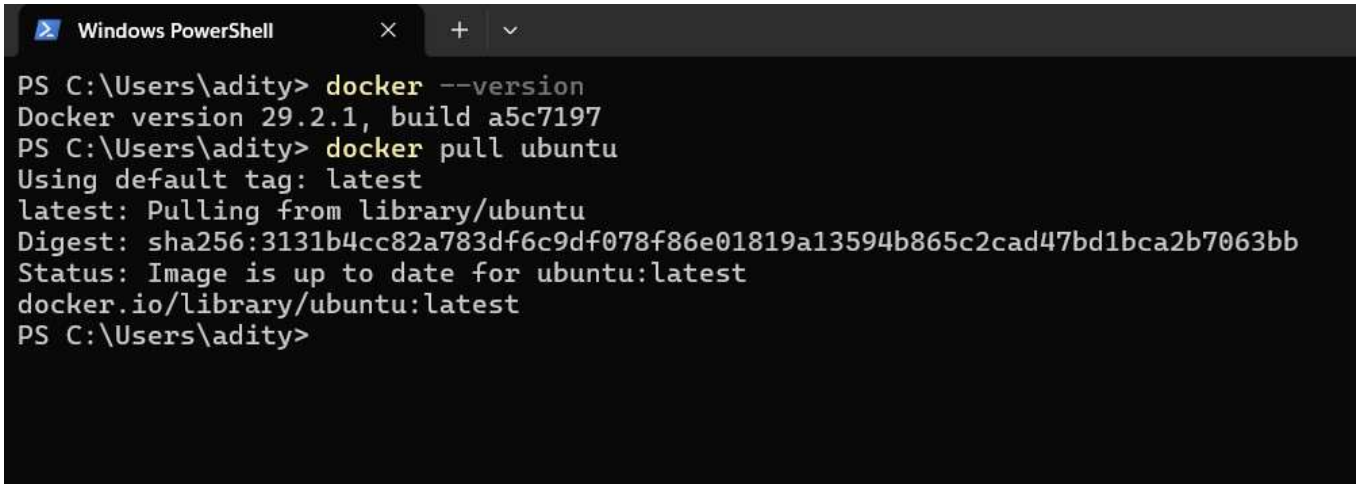


Output:

Open Command Prompt, run the following commands:

1) To check the version of Docker: docker --version

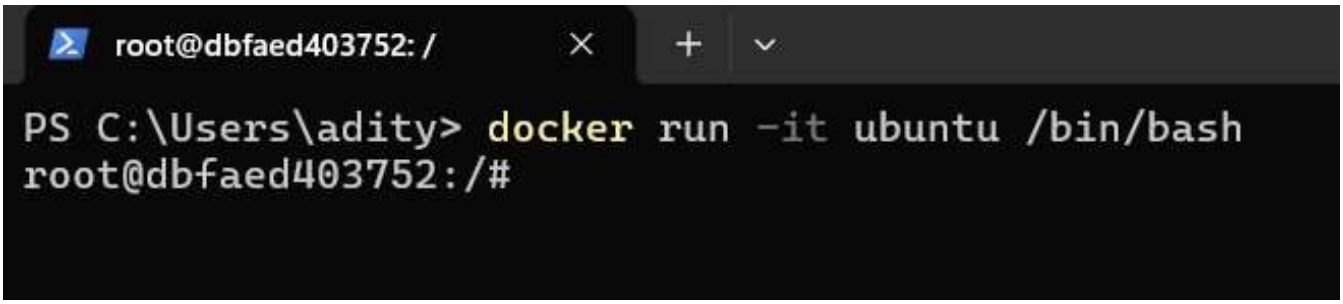
2) To install image of ubuntu docker pull ubuntu



3) Check downloaded images, docker images



4) Run ubuntu OS docker run -it ubuntu /bin/bash

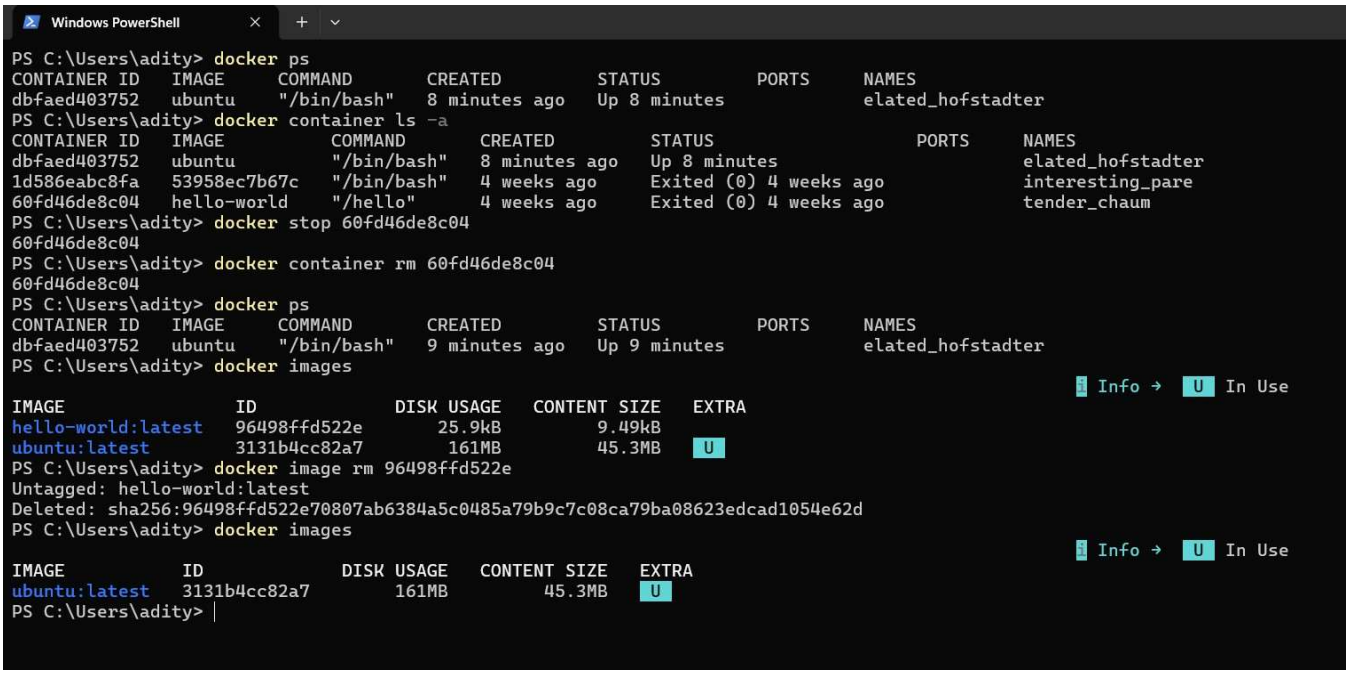


5) Open another Command Prompt and follow the steps shown below.

-docker ps docker container ls –a docker container rm b71e3e6b1118 //copy docker id for remove but first (Use your container ID in the above command)

stop your docker

- docker container stop b71e3e6b1118
- docker container rm b71e3e6b1118
- docker ps
- docker //list all docker commands
- docker images
- docker image rm ff0fea8310f3 // copy image id from previous output (Use your image ID in the above command)
- docker run -it ubuntu /bin/bash //check output



Conclusion :

Q1. What is the difference between containerization and virtualization?

ANS - **Containerization** is a technology that allows applications and their dependencies to run in isolated environments called **containers** while sharing the host operating system's kernel. Containers are lightweight, start quickly, and require fewer system resources. **Virtualization**, on the other hand, creates complete virtual machines (VMs) using a **hypervisor**, where each virtual machine has its own operating system, applications, and allocated hardware resources. Therefore, virtual machines are generally heavier and take more time to start compared with containers. Docker is a popular platform used for containerization, while software such as VMware and VirtualBox is commonly used for virtualization.

Q2. What is Docker Daemon?

ANS - The **Docker Daemon**, also known as **dockerd**, is a background service responsible for managing Docker objects such as **containers, images, networks, and volumes**. It listens for requests from the Docker client through the Docker API and performs the required operations. For example, when a user executes a command such as **docker run ubuntu**, the Docker client sends the request to the Docker Daemon, which finds or downloads the required image and creates and starts the container. In simple terms, the Docker Daemon is the **core background process that performs and manages Docker operations**.